4. Documentación de la API — Órbita

Dato	Valor
Fuente	Código de `orbita-web/sql/`, `orbita-web/supabase/functions/`, `orbita-web/api/` y `orbita-movil/app/.../data/Repo.kt`; firmas verificadas contra la base en producción (11 nov 2026)
Contrato OpenAPI	`orbita-api/openapi.yaml` (endpoints principales)
URL base	`https://<proyecto>.supabase.co` (valor real en las variables de entorno del proyecto)

Estado de verificación

- ✅ Las firmas de las funciones listadas existen en producción.
- ⚠️ No hay pruebas automatizadas de la API ni colecciones de pruebas (Postman u otras). Los ejemplos son ilustrativos y no son respuestas capturadas.
- ⚠️ Las funciones de evidencia (`register_appointment_file`, `delete_appointment_file`) y el bucket `appointment-evidence` **no existen todavía en producción**.

API REST que usan la web de Órbita y la app Android **Órbita Citas**. Sirve para coordinar citas (grabaciones, reuniones, entrevistas y accesos a cuentas) y para el registro de asistencia del equipo.

La API no es un servidor aparte: es la capa REST de **Supabase** (PostgREST, Auth y Storage) sobre la base de datos de Órbita. Las reglas viven en la base:

- **Permisos por fila (RLS):** cada persona solo lee lo que le corresponde.
- **Funciones `security definer`:** son la única forma de cambiar datos de citas. Validan el rol, el estado de la cita y los empalmes.

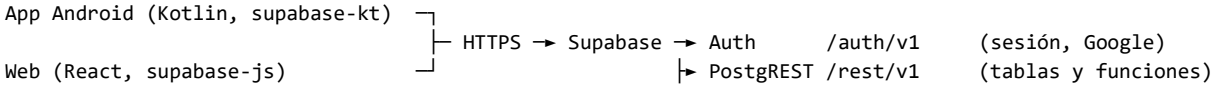
Así ningún cliente, ni la web, ni la app, ni alguien con la clave pública, puede saltarse una regla.

Contrato en formato OpenAPI 3: `openapi.yaml`.

Índice

1. Arquitectura
2. Autenticación
3. Convenciones
4. Lecturas (tablas)
5. Citas: funciones
6. Evidencia (archivos)
7. Asistencia
8. Avisos y procesos automáticos
9. Errores
10. Flujo completo de ejemplo

1. Arquitectura



```
├► Storage   /storage/v1  (evidencia privada)
└► PostgreSQL: RLS, funciones, triggers y pg_cron
```

Pieza	Para qué
`/auth/v1`	Iniciar sesión con correo y contraseña o con Google (token de ID). Devuelve un JWT.
`/rest/v1/<tabla>`	Leer citas, participantes, acuerdos, tipos, lugares, avisos. Solo lectura para la app.
`/rest/v1/rpc/<función>`	Todas las acciones: solicitar, programar, responder, iniciar, cerrar, etc.
`/storage/v1`	Subir y abrir fotos o PDF de evidencia en un bucket privado.
`pg_cron`	Recordatorios de citas cada 10 minutos y revisiones de asistencia.

Roles:

Rol	Puede
`admin` / superadmin	Coordinar citas: programar, reprogramar, iniciar, cerrar, registrar asistencia y acuerdos, subir evidencia, agregar tipos y lugares. Ver toda la asistencia.
`team` (equipo)	Solicitar citas, responder invitaciones, registrar su entrada y salida. Ve las citas donde participa.
`client` (cliente)	Solicitar citas de **sus** proyectos, responder invitaciones. Solo ve sus citas y lo marcado como visible para el cliente.

Las cuentas solo se crean por invitación desde la web. Un correo sin invitación no puede registrarse, ni con Google.

2. Autenticación

Todas las peticiones llevan dos encabezados:

```
apikey: <CLAVE_PUBLICA>                  # clave pública (anon) del proyecto
Authorization: Bearer <ACCESS_TOKEN>     # JWT del usuario después de iniciar sesión
```

La clave pública no da acceso a datos por sí sola: sin sesión, las tablas devuelven 0 filas y las funciones se niegan.

Correo y contraseña

```
POST /auth/v1/token?grant_type=password
Content-Type: application/json

{ "email": "persona@ejemplo.com", "password": "********" }
```

Respuesta: `access_token` (JWT, 1 h), `refresh_token`, `user`.

Google (app Android)

La app obtiene un token de ID con Credential Manager, enviando el **hash SHA-256 de un nonce**, y lo cambia por una sesión:

```
POST /auth/v1/token?grant_type=id_token
{ "provider": "google", "id_token": "<token de Google>", "nonce": "<nonce sin hash>" }
```

Renovar sesión

```
POST /auth/v1/token?grant_type=refresh_token
{ "refresh_token": "<refresh_token>" }
```

Perfil y licencia

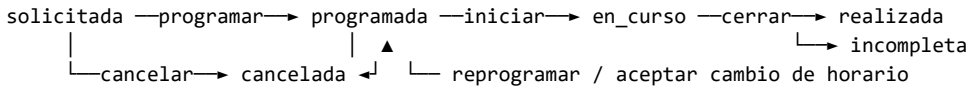
Después de entrar, la app lee el perfil y la licencia. Si la cuenta está desactivada o la licencia suspendida, cierra la sesión.

```
GET /rest/v1/users?id=eq.<uid>&select=id,email,full_name,role,active,is_super_admin
GET /rest/v1/app_license?id=eq.1&select=status
```

3. Convenciones

- **Fechas:** ISO 8601 con zona (`2026-11-12T10:00:00-06:00`). Se guardan en UTC y se muestran en `America/Mexico_City`.
- **Funciones:** `POST /rest/v1/rpc/<nombre>` con un JSON cuyos nombres son los parámetros (`p_...`). Los parámetros con valor por omisión se pueden omitir.
- **Horarios propuestos (`slots`):** lista de 1 a 3 objetos `{ "starts_at": "...", "ends_at": "..." }`, futuros y de 12 h como máximo.
- **Filtros de lectura:** sintaxis de PostgREST (`?status=eq.programada&order=starts_at.asc`).
- **Respuestas de error:** JSON `{ "code": "P0001", "message": "<motivo en español>" }`. Ver Errores.

Estados de una cita



Respuestas de un invitado

`pendiente` → `aceptada` | `rechazada` (con motivo) | `cambio_solicitado` (con 1 a 3 horarios).

4. Lecturas (tablas)

Solo lectura (`GET`). Los permisos por fila deciden qué filas ve cada persona.

Recurso	Quién ve	Ejemplo
`appointments`	Admins: todas. Los demás: donde participan o que pidieron.	`GET /rest/v1/appointments?select=*,project:projects(id,name),participants:appointment_participants(*,user:users(id,full_name,role))&order=starts_at.asc`
`appointment_participants`	Igual que la cita.	`GET /rest/v1/appointment_participants?appointment_id=eq.<id>`
`appointment_agreements`	Admins y equipo invitado: todos. Clientes: solo `visible_to_client = true`.	`GET /rest/v1/appointment_agreements?appointment_id=eq.<id>`
`appointment_events`	Admins y equipo invitado (historial).	`GET /rest/v1/appointment_events?appointment_id=eq.<id>&order=created_at`
`appointment_files`	Igual que los acuerdos.	`GET /rest/v1/appointment_files?appointment_id=eq.<id>`
`appointment_kinds`	Todos (tipos activos).	`GET /rest/v1/appointment_kinds?order=sort_order`

appointment_places	Todos (estudios activos).	GET /rest/v1/appointment_places?active=eq.true
notifications	Cada persona, las suyas.	GET /rest/v1/notifications?user_id=eq.<uid>&order=created_at.desc&limit=60
projects	Admins y equipo: todos. Clientes: los suyos.	GET /rest/v1/projects?select=id,name,recording_mode,launch_date

Marcar un aviso como leído es la única escritura directa permitida:

```
PATCH /rest/v1/notifications?id=eq.<id>
{ "read": true }
```

Ejemplo de cita (respuesta):

```
{
  "id": "5b1f…",
  "kind": "grabacion",
  "title": "Sesión de grabación VSL",
  "status": "programada",
  "modality": "presencial",
  "location": "Bunker",
  "location_kind": "estudio",
  "address": "Av. Ejemplo 123, Guadalajara",
  "latitude": 20.67, "longitude": -103.35,
  "meeting_url": null,
  "starts_at": "2026-11-12T16:00:00+00:00",
  "ends_at": "2026-11-12T18:00:00+00:00",
  "timezone": "America/Mexico_City",
  "proposed_slots": [],
  "related_task_title": null,
  "participants": [
    { "user_id": "…", "role_label": "Coordinador", "required": true, "response": "aceptada" },
    { "user_id": "…", "role_label": "Cliente", "required": true, "response": "pendiente" }
  ]
}
```

5. Citas: funciones

Todas se llaman con `POST /rest/v1/rpc/<nombre>`.

5.1 request_appointment: solicitar una cita

Quién: cualquier usuario activo. Un cliente, solo para sus proyectos. No reserva horario.

Parámetro	Tipo	Notas	
p_project_id	uuid \	null	Obligatorio para clientes.
p_kind	text	Clave de appointment_kinds (reunion, grabacion, entrevista, accesos o uno nuevo).	
p_reason	text	Motivo, 3 a 2000 caracteres.	
p_slots	json	1 a 3 horarios propuestos.	
p_location_kind	text	**Obligatorio si el tipo es presencial:** estudio o cliente.	

p_place_id	uuid	Opcional: estudio elegido. Sin él, la agencia decide.	
p_address	text	Con cliente: dirección (5+ caracteres) o coordenadas.	
p_lat, p_lng	number	Ubicación actual del teléfono (opcional).	

```
{
  "p_project_id": "…", "p_kind": "grabacion", "p_reason": "Grabar el VSL con el guion aprobado.",
  "p_slots": [{ "starts_at": "2026-11-12T10:00:00-06:00", "ends_at": "2026-11-12T12:00:00-06:00" }],
  "p_location_kind": "cliente", "p_address": "Calle Ejemplo 45, Zapopan", "p_lat": 20.72, "p_lng": -103.39
}
```

Devuelve: "<uuid de la cita>". Avisa a todos los admins.

5.2 schedule_appointment: programar o reprogramar (admin)

Crea una cita nueva (p_appointment_id = null) o programa una solicitud o reprograma una existente.

Parámetro	Tipo	Notas	
p_appointment_id	uuid \	null	null para una cita nueva.
p_project_id	uuid \	null	null = cita interna.
p_kind, p_title	text		
p_starts, p_ends	timestamptz	Futuro; máximo 12 h.	
p_modality	text	presencial o virtual.	
p_location	text	Nombre del lugar cuando p_location_kind = otro.	
p_meeting_url	text	Con virtual: enlace https:// de Meet o Zoom.	
p_participants	json	[{ "user_id": "…", "required": true, "role_label": "Cliente" }]. El coordinador se agrega solo.	
p_related_task_id	uuid	Tarea de referencia (opcional).	
p_location_kind	text	Con presencial: estudio (requiere p_place_id), cliente (requiere dirección o coordenadas) u otro.	
p_place_id, p_address, p_lat, p_lng		Lugar.	

Regla de empalmes: si un invitado **obligatorio** ya tiene otra cita en ese horario, responde CONFLICTO: <nombres> ya tiene otra cita en ese horario. Una restricción de exclusión en la base lo garantiza aunque dos admins publiquen al mismo tiempo. Con invitados opcionales solo se avisa en la app.

Devuelve: uuid de la cita. Avisa a los invitados ("🗓️ Nueva cita" o "🔁 Cita reprogramada").

5.3 check_appointment_conflicts: ¿quién está ocupado? (admin)

```
{ "p_starts": "…", "p_ends": "…", "p_user_ids": ["…", "…"], "p_exclude_appointment": null }
```

Devuelve: [{ "user_id", "full_name", "appointment_id", "appointment_title", "required_there" }]. La app lo usa para mostrar **Libre / Ocupado**.

5.4 respond_appointment: responder como invitado

p_response	Requiere

aceptada	—
rechazada	p_note (motivo, 3+ caracteres)
cambio_solicitado	p_slots (1 a 3 horarios) y p_note opcional

```
{ "p_appointment_id": "…", "p_response": "cambio_solicitado", "p_note": "¿Lo movemos al jueves?",
  "p_slots": [{ "starts_at": "…", "ends_at": "…" }] }
```

Solo con la cita programada. Avisa al coordinador.

5.5 resolve_change_request: aceptar o rechazar un cambio de horario (admin)

```
{ "p_appointment_id": "…", "p_user_id": "<quien pidió el cambio>", "p_accept": true, "p_slot_index": 0, "p_note": null }
```

- p_accept = true: la cita pasa al horario propuesto p_slot_index. Quien lo pidió queda aceptada y los demás vuelven a confirmar. Revisa empalmes.
- p_accept = false: la persona vuelve a pendiente y recibe p_note.

5.6 cancel_appointment

{ "p_appointment_id": "…", "p_reason": "…" }. Lo hace un admin, o quien la pidió mientras sigue solicitada. Libera los horarios y avisa.

5.7 start_appointment (admin)

{ "p_appointment_id": "…" }. Pasa de programada a en_curso, desde una hora antes del inicio.

5.8 set_appointment_attendance (admin)

{ "p_appointment_id": "…", "p_user_id": "…", "p_attendance": "presente" | "ausente" }. Solo con la cita en curso.

5.9 add_appointment_agreement y delete_appointment_agreement (admin)

```
{ "p_appointment_id": "…", "p_body": "Regrabar la toma de cierre.", "p_visible_to_client": false }
```

Devuelve el uuid del acuerdo. Para borrar: { "p_agreement_id": "…" }.

5.10 close_appointment (admin)

{ "p_appointment_id": "…", "p_status": "realizada" | "incompleta", "p_note": "…" }. incompleta requiere nota. Libera horarios y avisa.

5.11 Tipos de cita (admin)

- add_appointment_kind: { "p_label": "Sesión de fotos", "p_default_modality": "presencial" }. Devuelve la clave del tipo.
- set_appointment_kind_active: { "p_slug": "…", "p_active": false } oculta un tipo sin tocar las citas que lo usan.

Por omisión, grabacion es **presencial**; reunion, entrevista y accesos son **virtuales**.

5.12 Lugares de la agencia (admin)

- add_appointment_place: { "p_name": "Bunker", "p_address": "…", "p_lat": 20.67, "p_lng": -103.35 }. Devuelve uuid.
- set_appointment_place_active: { "p_place_id": "…", "p_active": false }.

6. Evidencia (archivos)

Las fotos y PDF viven en el bucket **privado** `appointment-evidence`, con la ruta `<id de la cita>/<nombre único>`. Tamaño máximo: 10 MB. Tipos aceptados: JPEG, PNG, WEBP, HEIC y PDF.

11. **Subir** (admin, cita en curso o cerrada):

    ```http
    POST /storage/v1/object/appointment-evidence/<cita>/<uuid>.jpg
    Content-Type: image/jpeg
    <bytes>
    ```

12. **Registrar:** `POST /rest/v1/rpc/register_appointment_file`

    ```json
    { "p_appointment_id": "…", "p_path": "<cita>/<uuid>.jpg", "p_file_name": "toma1.jpg",
      "p_mime_type": "image/jpeg", "p_size_bytes": 482113, "p_visible_to_client": true }
    ```

13. **Abrir:** enlace temporal (10 min):

    ```http
    POST /storage/v1/object/sign/appointment-evidence/<ruta>
    { "expiresIn": 600 }
    ```

    Un cliente solo puede abrir archivos marcados como visibles para el cliente de citas donde participa.

14. **Borrar** (admin): `rpc/delete_appointment_file` `{ "p_file_id": "…" }` devuelve la ruta, y luego `DELETE /storage/v1/object/appointment-evidence/<ruta>`.

7. Asistencia

Función	Quién	Qué hace	
`registrar_entrada` `{ "p_location": "oficina" \`	"home_office" }`	equipo y admins	Registra la entrada con la hora del servidor y calcula el retardo según su horario.
`registrar_salida` {}	equipo y admins	Cierra las sesiones abiertas del día y calcula horas regulares y extra.	
`mi_sesion_hoy` {}	equipo y admins	Sesión abierta de hoy.	
`asistencia_rango` { "p_from": "2026-11-01", "p_to": "2026-11-07", "p_tolerancia": 5 }	admins: todos; los demás: lo suyo	Una fila por persona y día con `estado`: `a_tiempo`, `retardo`, `sin_registro`, `pendiente`, `descanso`,	

		dia_extra.	
admin_corregir_sesion { "p_session_id", "p_started_at", "p_ended_at", "p_reason" }	admins	Corrige una sesión y deja registro.	

Aviso en vivo: al registrar la **primera entrada del día**, un trigger avisa a los admins:

- 🟢 "Entrada a tiempo", hasta 5 min de tolerancia.
- 🟠 "Entrada con retardo", con los minutos.

El panel web "Hoy en vivo" se actualiza por Realtime con cada entrada o salida.

8. Avisos y procesos automáticos

Los avisos son filas de `notifications` (`type = 'appointment'` con `appointment_id`, o `type = 'attendance'`). La web los muestra en la campana. La app Android los revisa cada 15 minutos y los muestra como notificación, con botones **Ver / Aceptar** en invitaciones y **Ver / Unirse** o **Cómo llegar** en recordatorios.

Proceso	Cuándo	Qué hace
`citas-recordatorios` (pg_cron)	cada 10 min	Recordatorios 24 h y 1 h antes de cada cita programada, sin repetir.
`aviso_entrada` (trigger)	al registrar entrada	Aviso verde o naranja a los admins.
`checador-revisar-asistencia` (pg_cron)	cada 15 min, 7:00–14:45 L-S	Avisa si alguien no ha registrado entrada 30 min después de su horario.
`checador-resumen-diario` (pg_cron)	9:30 L-S	Resumen de asistencia a los admins.
`checador-cerrar-olvidadas` (pg_cron)	00:15	Cierra sesiones que nadie cerró.

9. Errores

HTTP	Cuándo	Ejemplo de `message`
400	Una regla de negocio no se cumple (`code: P0001`)	`Solo un administrador puede programar citas · Propón de 1 a 3 horarios · Escribe el motivo del rechazo · Pega el enlace de Meet o Zoom (debe empezar con https://) · Indica si la cita es en el estudio o en tu domicilio`
400	Empalme	`CONFLICTO: Ana López ya tiene otra cita en ese horario`
401	Sin sesión o token vencido	`JWT expired`
403 / 0 filas	Sin permiso por fila	La lectura devuelve `[]`
404	Función o recurso inexistente	`Could not find the function …`

Los mensajes ya están en español para mostrarlos tal cual. La app quita el prefijo `CONFLICTO:` .

10. Flujo completo de ejemplo

```
URL=https://<proyecto>.supabase.co
KEY=<CLAVE_PUBLICA>

# 1. Cliente inicia sesión
TOKEN=$(curl -s "$URL/auth/v1/token?grant_type=password" -H "apikey: $KEY" \
  -H "Content-Type: application/json" -d '{"email":"cliente@ejemplo.com","password":"********"}' | jq -r
.access_token)
```

```
# 2. Solicita una entrevista virtual
curl -s "$URL/rest/v1/rpc/request_appointment" -H "apikey: $KEY" -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"p_project_id":"<proyecto>","p_kind":"entrevista","p_reason":"Dar acceso a Meta Business",
       "p_slots":[{"starts_at":"2026-11-16T17:00:00-06:00","ends_at":"2026-11-16T17:30:00-06:00"}]}'

# 3. Un admin la programa con enlace de Meet (con su propio token)
curl -s "$URL/rest/v1/rpc/schedule_appointment" -H "apikey: $KEY" -H "Authorization: Bearer $TOKEN_ADMIN" \
  -H "Content-Type: application/json" \
  -d '{"p_appointment_id":"<cita>","p_project_id":"<proyecto>","p_kind":"entrevista","p_title":"Acceso a Meta
Business",
       "p_starts":"2026-11-16T17:00:00-06:00","p_ends":"2026-11-16T17:30:00-06:00","p_modality":"virtual",
       "p_location":null,"p_meeting_url":"https://meet.google.com/abc-defg-hij",
       "p_participants":[{"user_id":"<cliente>","required":true,"role_label":"Cliente"}]}'

# 4. El cliente acepta
curl -s "$URL/rest/v1/rpc/respond_appointment" -H "apikey: $KEY" -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" -d '{"p_appointment_id":"<cita>","p_response":"aceptada"}'

# 5. El día de la cita (admin; mismos encabezados que el paso 3)
curl -s "$URL/rest/v1/rpc/start_appointment" … -d '{"p_appointment_id":"<cita>"}'
curl -s "$URL/rest/v1/rpc/set_appointment_attendance" … -d
'{"p_appointment_id":"<cita>","p_user_id":"<cliente>","p_attendance":"presente"}'
curl -s "$URL/rest/v1/rpc/add_appointment_agreement" … -d '{"p_appointment_id":"<cita>","p_body":"Enviar acceso
de socio","p_visible_to_client":true}'
curl -s "$URL/rest/v1/rpc/close_appointment" … -d '{"p_appointment_id":"<cita>","p_status":"realizada"}'
```

Código fuente

- Definición de la API (SQL): `orbita-web/sql`, archivos `2026-11-09_citas.sql`, `2026-11-10_citas_v2_y_asistencia.sql`, `2026-11-10_evidencia_citas.sql`, `2026-11-11_ubicacion_citas.sql` y `2026-10-23_checador_servidor.sql`.
- Cliente Android: `orbita-movil`, archivo `data/Repo.kt`.
- Cliente web: `orbita-web`.

11. Otros endpoints (fuera de PostgREST)

11.1 Edge Function send-user-invitation ☑ publicada

Dato	Valor
Método y ruta	`POST /functions/v1/send-user-invitation`
Autenticación	JWT de un usuario **administrador activo o superadmin** (`Authorization: Bearer`). Supabase verifica el JWT.
Cuerpo	`{ "email": "persona@ejemplo.com", "fullName": "Nombre", "roleLabel": "cliente", "inviteUrl": "https://…?invite=<id>" }`
Respuestas	`200 { "success": true }` · `403 { "error": "Solo un administrador puede invitar usuarios" }` · `400 { "error": "<motivo>" }` (correo inválido, enlace sin https, SMTP sin configurar o rechazado)
Implementación	`orbita-web/supabase/functions/send-user-invitation/index.ts` (SMTP de Gmail, puerto 465)

11.2 Edge Function enviar-push ☑ publicada

Dato	Valor
Método y ruta	`POST /functions/v1/enviar-push`
Autenticación	Se publica **sin verificar JWT**. La llama el trigger `enviar_push` de la tabla `notifications` mediante `pg_net`.

Cuerpo	`{ "notification_id": "<uuid>" }`
Respuestas	`200 { "enviado": true, "enviados": n }` · `200 { "enviado": false, "motivo": "ya enviado, viejo o inexistente" }` · `400 { "error": "Aviso no válido" }` · `500 { "error": "No se pudo enviar" }`
Garantías	Solo envía avisos de menos de 10 minutos y los marca con `pushed_at`, así que repetir la llamada no reenvía. Borra los tokens que FCM reporta como inválidos.
Implementación	`orbita-web/supabase/functions/enviar-push/index.ts` (FCM HTTP v1, firma RS256 con la cuenta de servicio)

11.3 Función de Vercel /api/check-spelling ⚠ no operativa

Dato	Valor
Método y ruta	`POST https://orbita-web-ten.vercel.app/api/check-spelling`
Autenticación	`Authorization: Bearer <JWT de Supabase>`, validado contra `/auth/v1/user`
Cuerpo	`{ "url": "https://pagina-publica…" }`
Respuestas	`200 { "result": "…", "chars": n }` · `400` (falta URL, página privada o sin texto) · `401` (sin sesión) · `405` (método) · `500`
Estado	Requiere `GEMINI_API_KEY` en Vercel, que no está configurada según las notas del proyecto. Bloquea direcciones privadas o locales (protección SSRF).

11.4 Realtime

Los clientes se suscriben a cambios de `notifications` (campana web), `work_sessions` (asistencia en vivo), `direct_messages`, `task_requests`, `appointments` y `appointment_participants`. La suscripción respeta RLS.

12. Correspondencia con el cliente Android

Todas las llamadas de la app están en `orbita-movil/app/src/main/java/com/orbitasinertegia/orbita/data/Repo.kt`.

Método en Repo	Endpoint
`signIn`, `signInWithGoogle`, `resetPassword`	`/auth/v1/token`, `/auth/v1/recover`
`appointments`, `appointment`	`GET /rest/v1/appointments` (con participantes, proyecto y solicitante)
`requestAppointment`, `scheduleAppointment`, `respond`, `resolveChange`, `cancel`, `start`, `setAttendance`, `addAgreement`, `deleteAgreement`, `close`	`POST /rest/v1/rpc/<función>`
`kinds`, `addKind`, `places`, `addPlace`	`appointment_kinds`, `appointment_places`, `rpc/add_appointment_kind`, `rpc/add_appointment_place`
`uploadEvidence`, `downloadEvidence`, `evidenceUrl`, `deleteEvidence`	Storage `appointment-evidence` + RPC de evidencia (⚠ sin aplicar en BD)
`notices`, `markRead`, `markAllRead`	`notifications`
`registerDeviceToken`, `unregisterDeviceToken`	`rpc/register_device_token`, `rpc/unregister_device_token`

La lista completa de funciones de la base, con permisos de ejecución, está en el Documento de base de datos (§5.6).